Agentic AI :

1. INTRODUCTION TO AI AGENTS

1. What is an AI Agent ?

- An AI Agent is an intelligent system that can perceive its environment, reason, take actions and achieve a goal autonomously (using an LLM or other AI model).
- It can interact with tools, data and other systems to complete real-world tasks.

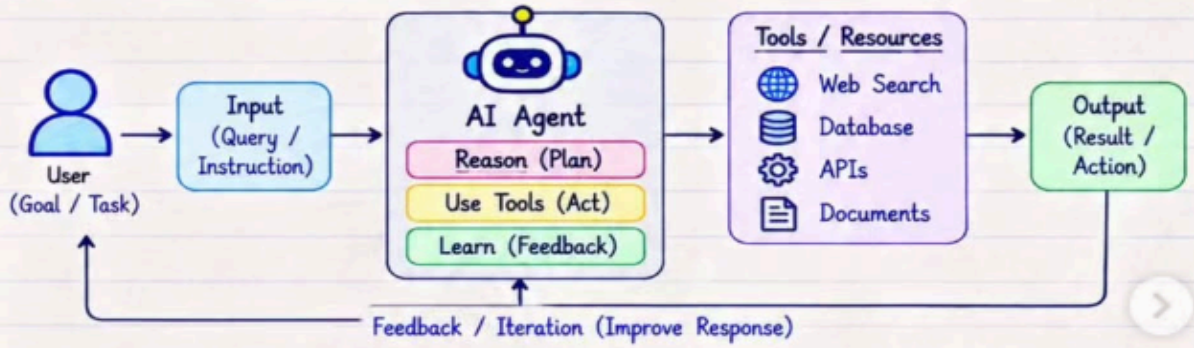
@code.withnarayan

2. Key Characteristics

- **Autonomous** : Works with minimal human input.
- **Goal-Oriented** : Performs tasks to achieve a goal.
- **Perception** : Understands input from user or environment.
- **Reasoning** : Plans and makes decisions.
- **Action** : Executes tasks using tools or APIs.
- **Adaptive** : Learns and improves from feedback.

3. How Does an AI Agent Work ?

@code.withnarayan



4. Components of an AI Agent

Component	Description
LLM / Brain	Understands, reasons and plans.
Memory	Stores past interactions, context and user preferences.
Tools	External capabilities (web search, APIs, database, code, etc.).
Planner	Breaks the goal into smaller steps.
Executor	Runs the planned steps using tools.

5. Real-World Applications

- Personal Assistant (e.g., email, calendar, tasks)
- Business Automation (e.g., report generation)
- Code Assistant (e.g., write, test and run code)
- Customer Support (e.g., intelligent chatbot)
- Research Assistant (e.g., search, summarize papers)
- E-commerce Agent (e.g., product search & compare)

6. Benefits of AI Agents

- Automates complex and multi-step tasks.
- Saves time and improves productivity.
- Works with real-time and external data.
- Can use multiple tools (APIs, web, databases).
- Personalized and context-aware responses.
- Useful in business, education and daily life.

@code.withnarayan

7. Practice Questions (MCQs)

1. What is the main characteristic of an AI Agent ?
   (a) Only generates text
   (b) Can take actions to achieve a goal
   (c) Works without any input
   (d) Only analyzes data

2. Which of the following is a component of an AI Agent ?
   (a) Compiler
   (b) Memory
   (c) Operating System
   (d) Web Browser

2. AI Agents vs Traditional AI vs Generative AI

1. Overview

- AI has evolved from rule-based systems to generative models and now to AI Agents.
- Each type has a different purpose, capability and level of autonomy.

@code.withnarayan

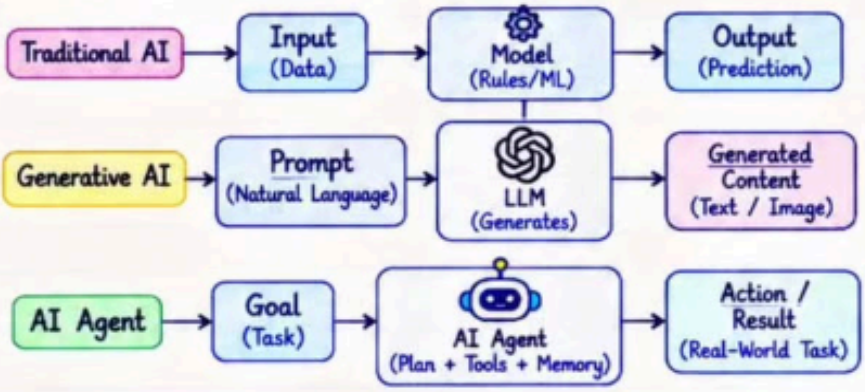
2. What are They ?

- Traditional AI : Rule-based or pre-trained systems for specific, predefined tasks.
- Generative AI : Uses large models (e.g., LLMs) to generate new content (text, image, code, etc.).
- AI Agent : An autonomous system that uses LLMs + tools + memory to achieve a goal.

3. Key Comparison

Aspect	Traditional AI	Generative AI	AI Agent
Main Goal	Solve specific, predefined problems	Generate new content (text, image, code, etc.)	Achieve a goal autonomously using tools and reasoning
Input	Structured or fixed data	Natural language (prompts)	Goal, instruction and real-time data
Output	Prediction / classification (e.g., spam or not)	Generated content (e.g., essay, image, code)	Action + result (e.g., book a ticket, send email)
Autonomy	Low (rule-based)	Medium (generates responses)	High (plans, decides and executes)
Use of Tools	No (or very limited)	Limited (mostly content generation)	Yes (APIs, database, web, etc.)
Memory	Usually not present	Limited (conversation context)	Yes (short-term & long-term)
Examples	Spam filter, loan approval	ChatGPT, DALL·E, Gemini	Travel agent, research agent, code assistant

4. Working Flow Comparison



5. When to Use ?

- Use Traditional AI when you need a simple and specific solution (e.g., classification, prediction).
- Use Generative AI when you want to create new content (e.g., text, image, code, summary).
- Use AI Agent when the task requires multiple steps, tool usage and autonomous execution (e.g., research, booking, automation).

@code.withnarayan

6. Key Takeaways

- Traditional AI → Task-specific, rule or model based.
- Generative AI → Creates new content using large models (LLMs).
- AI Agent → Autonomous, goal-driven system that can plan, use tools and act.
- AI Agents combine the power of LLMs, tools and memory to solve real-world problems.

7. Practice Questions (MCQs)

1. Which of the following is the key feature of an AI Agent ?
   (a) Only classifies data (b) Generates new content
   (c) Works autonomously to achieve a goal
   (d) Only follows fixed rules

2. Which is the correct order of evolution in AI ?
   (a) Generative AI → Traditional AI → AI Agent
   (b) Traditional AI → Generative AI → AI Agent
   (c) AI Agent → Generative AI → Traditional AI
   (d) Traditional AI → AI Agent → Generative AI

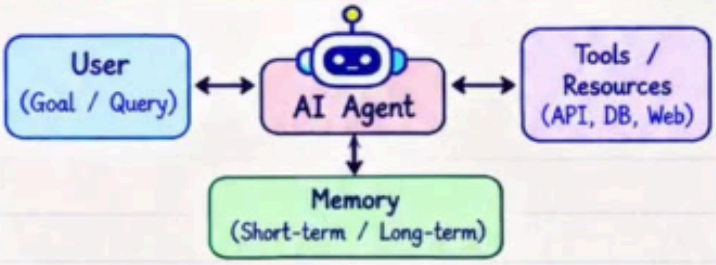
3. ARCHITECTURE OF AI AGENTS

1. What is AI Agent Architecture ?

- It is the overall structure that shows how different components of an AI Agent work together to perceive, reason, plan, use tools and take action to achieve a goal.

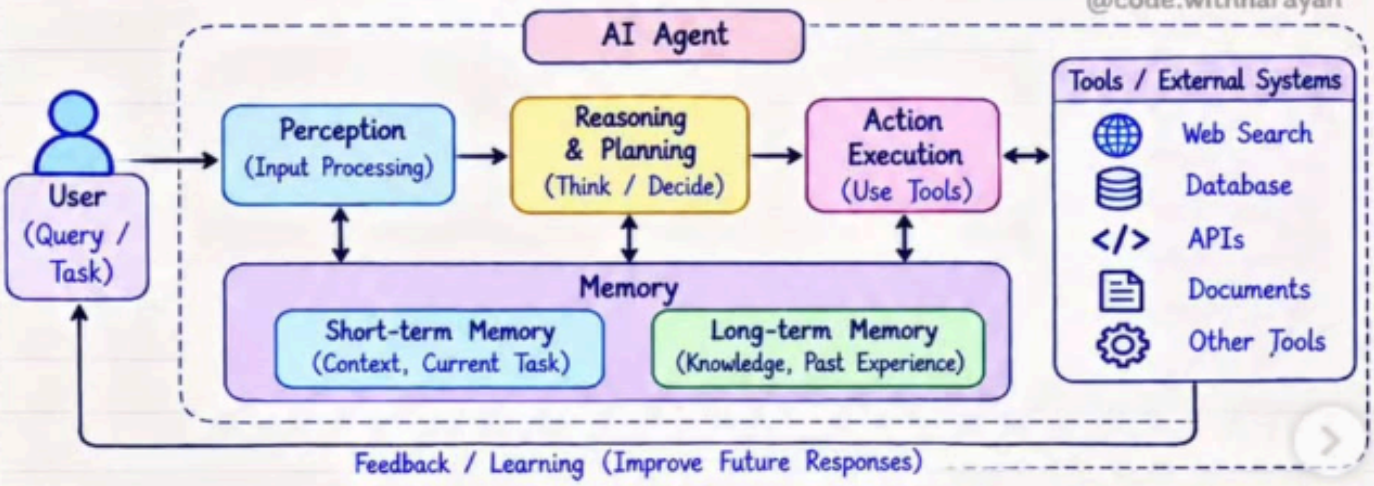
@code.withnarayan

2. High-Level Architecture (Overview)



3. Detailed Architecture of an AI Agent

@code.withnarayan



4. Core Components

Component	Description
Perception	Takes input from user or environment and understands it.
Reasoning & Planning	Analyzes the input, plans the steps and decides what to do.
Action Execution	Performs the planned actions using tools, APIs or other systems.
Memory	Stores context, past interactions and useful knowledge (short-term & long-term).
Tools	External capabilities like web search, database, file system, code execution, etc.
Feedback / Learning	Learns from results and improves future responses.

5. Architecture Flow (Step by Step)

1. Receive a goal or query from the user.
2. Perceive and understand the input.
3. Reason and create a plan (multi-step if needed).
4. Use tools (e.g., web search, API, database).
5. Take action and generate the final output.
6. Store important information in memory.
7. Use feedback to improve future responses.

6. Key Takeaways

- AI Agent = Perception + Reasoning + Action + Memory + Tools + Learning.
- Memory (short-term & long-term) makes the agent context-aware.
- Tools extend the agent's real-world capabilities.
- It works in a loop with feedback for continuous improvement.

7. Practice Questions (MCQs)

1. Which of the following is NOT a core component of an AI Agent ?
   (a) Perception (b) Reasoning & Planning
   (c) Compiler (d) Memory

2. What is the main role of Memory in an AI Agent ?
   (a) To compile the code (b) To store past interactions
   (c) To generate images (d) To translate language

@code.withnarayan

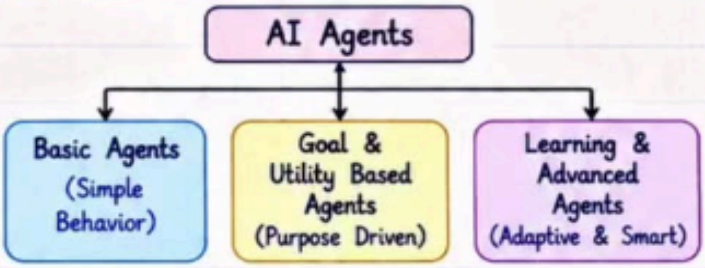
4. TYPES OF AI AGENTS

1. What are Types of AI Agents ?

- AI Agents can be classified into different types based on their capabilities, decision-making style, memory, and learning ability.
- Each type is suitable for different kinds of problems and applications.

@code.withnarayan

2. Classification Overview



3. Types of AI Agents (with Examples)

Type of Agent	Description	Example
1. Simple Reflex Agent	Takes action based on the current input using fixed rules.	Thermostat, Spam filter
2. Model-Based Reflex Agent	Uses an internal model of the environment to make better decisions.	Robot vacuum, Game agent
3. Goal-Based Agent	Chooses actions to achieve a specific goal.	Route planner (e.g., Google Maps)
4. Utility-Based Agent	Selects the best action by maximizing a utility (e.g., reward, performance, cost).	Recommendation system (e.g., Netflix)
5. Learning Agent	Improves its performance from past experience (learns from data).	Chatbot that gets better over time
6. Multi-Agent System	A group of agents that work together to achieve a common goal.	AI team for task automation (e.g., CrewAI, AutoGen)

4. Working of Each Type (Intuition)



@code.withnarayan

5. Key Differences

Aspect	Simple Reflex	Goal-Based	Utility-Based	Learning
Uses Memory	✗	✓	✓	✓
Has a Goal	✗	✓	✓	✓
Optimizes (Utility)	✗	✗	✓	✓
Learns from Experience	✗	✗	✗	✓

6. Real-World Examples

- Chatbot → Learning Agent
- Self-Driving Car → Utility-Based Agent
- Product Recommender → Goal/Utility Agent
- AI Team (Multi-Agent) → Multi-Agent System
- Game Playing AI → Model-Based Agent

7. Key Takeaways

- Different types of AI Agents are designed for different levels of intelligence and tasks.
- Simple → Rule-based, Goal-based → Purpose-driven, Utility-based → Best outcome, Learning → Improves over time.
- Multi-Agent systems enable collaboration among multiple agents.

@code.withnarayan

8. Practice Questions (MCQs)

1. Which type of agent learns from past experience ?
   (a) Simple Reflex Agent (b) Goal-Based Agent
   (c) Utility-Based Agent (d) Learning Agent
2. Which agent selects the best action by maximizing a utility function ?
   (a) Reflex Agent (b) Model-Based Agent
   (c) Utility-Based Agent (d) Multi-Agent System

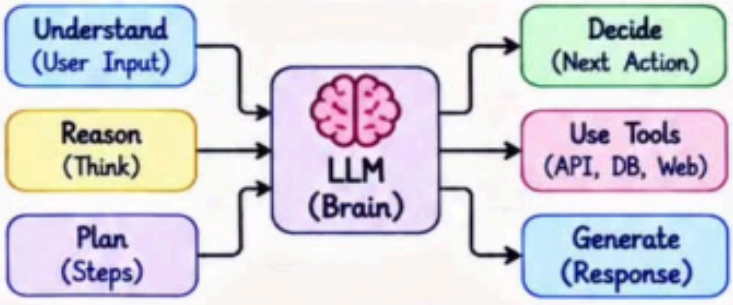
5. LLMs as the Brain of AI Agents

1. Why LLMs are the Brain ?

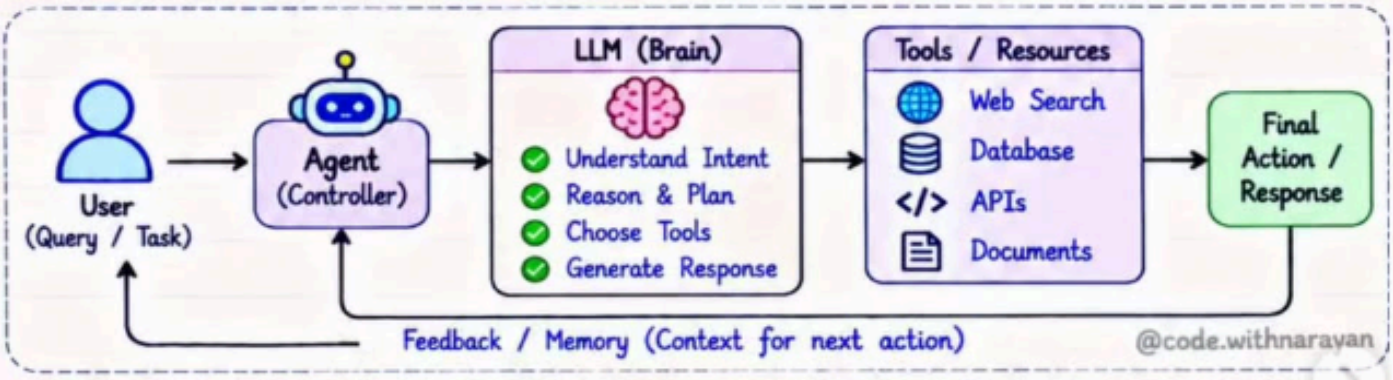
- LLMs (Large Language Models) act as the **brain** of an AI Agent because they **understand**, **reason**, **plan**, **decide** and **generate actions** in natural language.
- They provide the core intelligence that allows the agent to think and solve complex, multi-step tasks.

@code.withnarayan

2. Role of LLM in an AI Agent



3. LLM-Based AI Agent Architecture



@code.withnarayan

4. Key Capabilities Enabled by LLMs

- **Language Understanding** : Understands user goals and context.
- **Reasoning** : Breaks down complex tasks and finds a solution.
- **Planning** : Creates a step-by-step action plan.
- **Tool Use** : Decides which tool (API, DB, Web) to use and how.
- **Memory Handling** : Uses past context to give better and consistent results.
- **Natural Interaction** : Communicates in easy and human-like language.

5. Example : LLM as Agent Brain

User Query

"Book a train ticket to Delhi for tomorrow."

LLM (Brain) – Agent Thinking

1. **Understand** : Book train ticket.
2. **Plan** : Check availability → Select train → Book ticket.
3. **Use Tools** : Call Train API (IRCTC or similar).
4. **Action** : Book the ticket.
5. **Response** : "Your ticket is booked successfully."

6. Popular LLMs Used in AI Agents

LLM	Provider	Key Strength
GPT-4 / GPT-4o	OpenAI	Strong reasoning, tool use, multimodal.
Claude 3	Anthropic	Long context, safety, reliable.
Gemini 1.5	Google	Multimodal, large context window.
Llama 3	Meta	Open-source, cost-effective.
Mistral	Mistral AI	Fast, lightweight, good performance.

@code.withnarayan

7. Key Takeaways

- LLM is the core brain of an AI Agent.
- It helps the agent to understand, reason, plan and take actions.
- It works with tools, memory and external data.
- Popular LLMs: GPT-4, Claude, Gemini, Llama, Mistral.

8. Practice Questions (MCQs)

1. Why is an LLM considered the brain of an AI Agent ?
2. Which of the following is a popular LLM used in agents ?
   (a) GPT-4 (b) MySQL (c) Docker (d) Kafka

6. TOOLS & FUNCTION CALLING

1. What are Tools in AI Agents ?

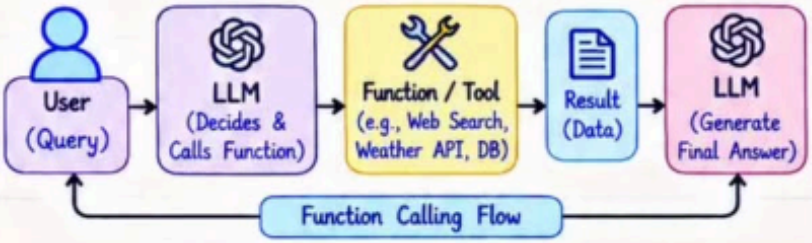
- Tools are external capabilities that an AI Agent can use to interact with the real world (e.g., search the web, access a database, run code, send an email, etc.).
- They extend the ability of an LLM to perform real and practical tasks beyond just generating text.

@code.withnarayan

2. What is Function Calling ?

- Function Calling is a feature that allows an LLM to call a predefined function (tool) with structured arguments (e.g., JSON) when needed.
- The model decides which function to call, passes the required input and uses the returned result to generate the final response.

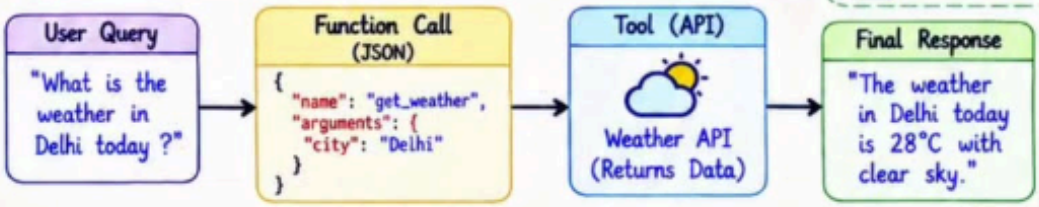
3. How Tools & Function Calling Work ?



4. Common Tools for AI Agents

- Web Search (e.g., Google, Bing)
- Database (e.g., PostgreSQL)
- Code Execution (e.g., Python)
- File Operations (read, write, upload)
- Email / Notification (send email)
- Productivity (calendar, to-do, etc.)

5. Function Calling Example (Concept)



@code.withnarayan

6. Python Example (OpenAI Function Calling)

Function Calling (Python)

```python
from openai import OpenAI
client = OpenAI()
tools = [{
    "type": "function",
    "function": {
        "name": "get_weather",
        "description": "Get current weather of a city",
        "parameters": {
            "type": "object",
            "properties": {"city": {"type": "string"}},
            "required": ["city"]
        }
    }
}]
response = client.chat.completions.create(
  model="gpt-4o",
  messages=[{"role": "user", "content": "Weather in Delhi?"}],
  tools=tools
)
```

@code.withnarayan

7. Benefits of Function Calling

- Lets LLMs perform real-world actions.
- Provides structured and reliable results.
- Connects with APIs, databases and custom logic.
- Makes AI Agents more useful and practical.
- Improves accuracy and reduces hallucination.

8. Key Takeaways

- Tools extend the skills of an AI Agent.
- Function Calling lets the LLM use these tools.
- It works using structured inputs (e.g., JSON).
- Common tools: web search, database, code, email, etc.

9. Practice Questions (MCQs)

1. What is the main purpose of Function Calling in an AI Agent ?
   - (a) To generate images
   - (b) To call external tools or functions
   - (c) To store user data
   - (d) To train a new model

2. Which of the following is an example of a tool used by an AI Agent ?
   - (a) Web Search API
   - (b) PostgreSQL Database
   - (c) Python Code Execution
   - (d) All of the above

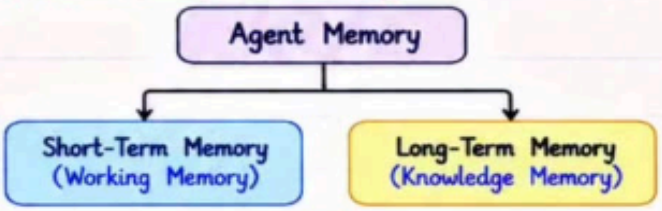
@code.withnarayan

7. AGENT MEMORY — SHORT-TERM & LONG-TERM MEMORY

1. What is Agent Memory ?

- Agent Memory is the mechanism that helps an AI Agent to **remember** information, **maintain context** and **use past experience** for better decisions.
- It gives the agent continuity and personalization across interactions.

2. Types of Agent Memory



@code.withnarayan

3. Short-Term Memory (STM)

- Stores temporary information during a single conversation or task.
- Helps to maintain the current context (e.g., previous messages, user intent, intermediate results).
- It is limited and gets cleared when the session ends.

Example (STM)

User : "Book a train ticket."

Agent : "From which city ?"

User : "Delhi to Mumbai."

The agent remembers "Book a ticket" and uses it to continue the conversation.

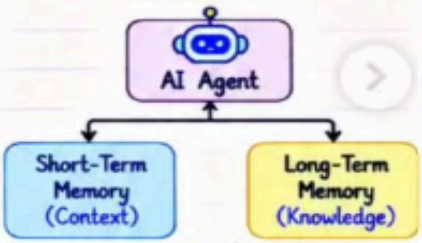
4. Long-Term Memory (LTM)

- Stores important and persistent information for a longer period.
- Includes user preferences, past interactions, domain knowledge, and learned experience.
- It helps in personalization and improves the quality of future responses.

5. Comparison : Short-Term vs Long-Term Memory

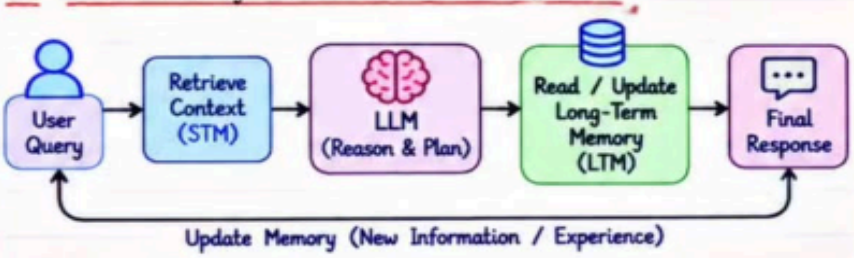
Aspect	Short-Term Memory (STM)	Long-Term Memory (LTM)
Purpose	Maintain current context	Store long-lasting knowledge
Duration	Temporary (session/task level)	Persistent (across sessions)
Capacity	Limited (few messages / tokens)	Large (documents, history, embeddings)
Examples	Recent chat messages, current inputs	User profile, past tasks, domain data
Clearing	Cleared after session ends	Remains until updated or deleted

6. Memory Architecture in AI Agent



@code.withnarayan

7. How Memory Works in an AI Agent ?



8. Real-World Examples

- **Chatbot** : Remembers user's past queries and preferences.
- **Travel Agent** : Saves past bookings and favorite routes.
- **Study Assistant** : Remembers the topics you are learning.
- **Customer Support Agent** : Stores user history and previous issues.

9. Key Takeaways

- Agent Memory makes the agent context-aware and more useful.
- Short-Term Memory (STM) → current, temporary context.
- Long-Term Memory (LTM) → persistent, long-lasting knowledge.
- Both together enable personalization, continuity and better decision making.

@code.withnarayan

10. Practice Questions (MCQs)

1. Which type of memory maintains the current conversation context ?
   (a) Long-Term Memory (b) Short-Term Memory
   (c) External Memory (d) None of the above

2. Which memory stores user preferences and past history ?
   (a) Short-Term Memory (b) Long-Term Memory
   (c) Working Memory (d) Cache Memory

8. PLANNING, REASONING & DECISION MAKING

1. Overview

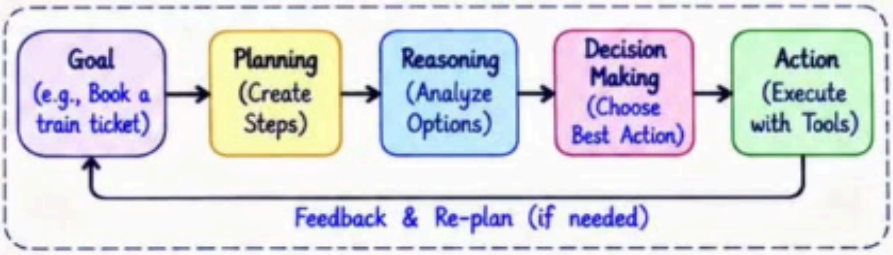
- Planning, Reasoning and Decision Making are the core cognitive abilities of an AI Agent that help it to achieve a goal autonomously.
- These work together to analyze a problem, create a plan, choose the best action and execute it.

@code.withnarayan

2. Key Concepts

Planning	Breaks a goal into smaller, step-by-step actions to achieve it.
Reasoning	Uses logic and knowledge to analyze, infer and solve a problem.
Decision Making	Selects the best action from available choices based on the current situation, goal and constraints.

3. How They Work Together ?



4. Planning in AI Agents

- Converts a high-level goal into a sequence of actionable steps.
- Common techniques :
  1. Chain-of-Thought (CoT)
  2. Task Decomposition
  3. Plan-and-Execute
  4. ReAct (Reason + Act)

5. Reasoning in AI Agents

- Helps the agent to think logically, understand context and draw useful conclusions.
- Types of Reasoning :
  1. Deductive Reasoning : General → Specific
  2. Inductive Reasoning : Specific → General
  3. Abductive Reasoning : Best possible explanation

6. Decision Making in AI Agents

- Chooses the most appropriate action using the plan, available tools, and current situation.
- Common Approaches :

@code.withnarayan

Approach	Idea	Example
Rule-Based	Follow predefined rules	If price < 1000, book ticket
Utility-Based	Choose action with highest benefit	Select cheapest & fastest option
Probabilistic	Decide under uncertainty	Choose option with highest probability of success
LLM-Based	Use LLM reasoning to decide	LLM compares options and picks the best one

7. Example : End-to-End Flow

Goal : "Book a train ticket from Delhi to Mumbai"

Planning	Break into steps: search train, compare options, book ticket.
Reasoning	Check availability, timing and price.
Decision Making	Choose the best train (e.g., fastest & cheapest).
Action	Call Train API and book the ticket.

Output : "Ticket booked successfully (PNR: 123456)"

8. Real-World Applications

- Travel Agent – Plan trip, choose best route and book tickets.
- Shopping Agent – Compare products and select the best deal.
- Coding Agent – Plan, reason and generate code solutions.
- Research Agent – Plan research, fetch data and summarize results.

9. Key Takeaways

- Planning creates a step-by-step roadmap to the goal.
- Reasoning provides logical thinking and problem solving.
- Decision Making selects the best action from available options.
- Together, they make the AI Agent autonomous, intelligent and goal-oriented.

@code.withnarayan

10. Practice Questions (MCQs)

1. Which component creates a step-by-step plan to achieve a goal ?
   (a) Reasoning (b) Planning
   (c) Decision Making (d) Memory
2. Which reasoning type finds the best possible explanation ?
   (a) Deductive (b) Inductive
   (c) Abductive (d) Probabilistic

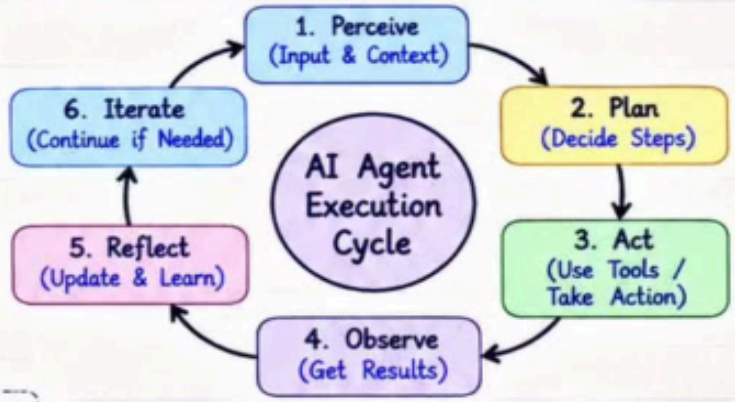
9. AGENT WORKFLOW & EXECUTION CYCLE

1. What is Agent Workflow ?

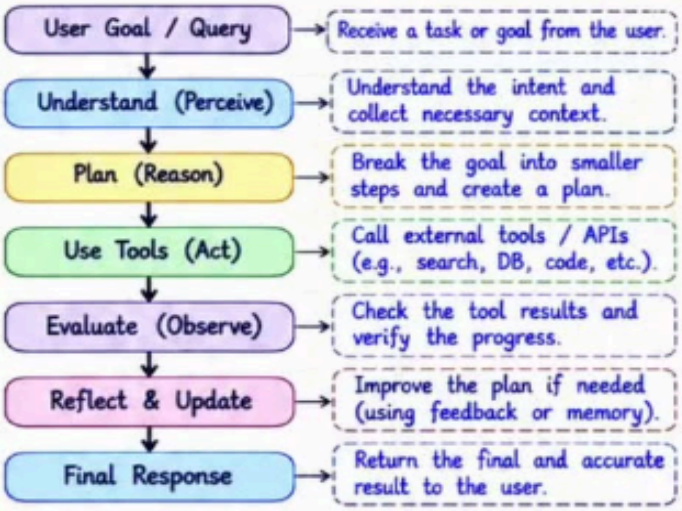
- Agent Workflow is the step-by-step process that an AI Agent follows to understand a goal, plan, use tools, take action and deliver the final result.
- It defines how different components (LLM, tools, memory, planner) work together in a continuous cycle.

@code.withnarayan

2. Agent Execution Cycle (Overview)



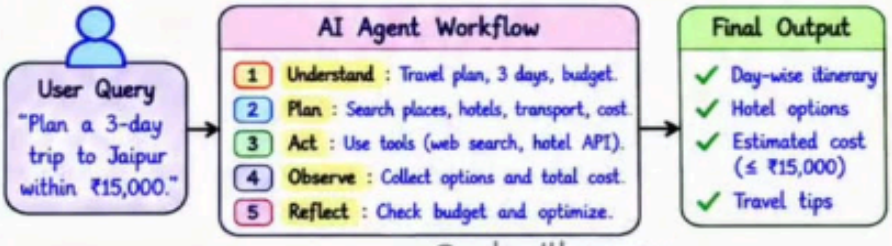
3. Detailed Agent Workflow



4. Key Stages of Execution Cycle

Stage	Purpose	Example
Perceive	Take input and understand the task.	"Find best laptop under ₹50,000."
Plan	Create a step-by-step plan.	Search → Compare → Shortlist → Recommend
Act	Use tools and take action.	Call web search / price API.
Observe	Analyze the results.	Get price, rating and features.
Reflect	Check if goal is achieved and improve if needed.	If not, refine search and try again.
Iterate	Repeat the cycle until the goal is completed.	Continue until best recommendation is found.

5. Example : End-to-End Workflow



@code.withnarayan

6. Common Workflow Patterns

1. **ReAct Workflow** : Reason + Act + Observe (iterative).
2. **Plan-and-Execute** : Plan first, then execute all steps.
3. **Self-Refine** : Generate → Critique → Improve.
4. **Multi-Agent Workflow** : Multiple agents work together (e.g., planner, researcher, writer).

7. Benefits of a Well-Defined Workflow

- Makes the agent more reliable and consistent.
- Helps in solving complex and multi-step tasks.
- Enables the use of external tools and real-time data.
- Supports iteration, error handling and self-improvement.
- Improves accuracy and the quality of the final result.

@code.withnarayan

8. Practice Questions (MCQs)

1. Which is the correct order in an AI Agent execution cycle ?
   (a) Plan → Act → Observe → Reflect
   (b) Perceive → Plan → Act → Observe → Reflect → Iterate
   (c) Act → Plan → Reflect → Observe
   (d) Observe → Act → Plan → Iterate

2. Which workflow pattern uses reasoning and tool execution in an iterative manner ?
   (a) Plan-and-Execute (b) ReAct
   (c) Self-Refine (d) Multi-Agent

@code.withnarayan

9. Key Takeaways

- Agent Workflow defines the complete flow from input to final result.
- The execution cycle is: Perceive → Plan → Act → Observe → Reflect → Iterate.
- Different workflow patterns (ReAct, Plan-and-Execute, Self-Refine) are used based on the task complexity.

10. AI AGENT FRAMEWORKS – LangChain, LangGraph, CrewAI & AutoGen

1. What are AI Agent Frameworks ?

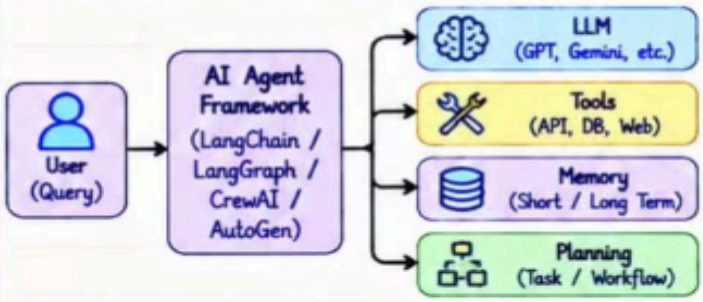
- AI Agent Frameworks are ready-to-use libraries and tools that help to build, run and manage AI Agents easily.
- They provide components like LLM integration, tools, memory, planning and multi-agent collaboration.

@code.withnarayan

2. Overview of Popular Frameworks

LangChain	• Popular framework to build LLM applications and AI Agents with tools, memory and more.
LangGraph	• A graph-based extension of LangChain for building stateful, multi-step and complex agents.
CrewAI	• A lightweight framework to create and manage multiple AI agents (a team) for a common goal.
AutoGen	• A multi-agent framework by Microsoft for building collaborative and conversational agents.

3. Architecture (High-Level View)



4. Key Features of Each Framework

Framework	Key Features
LangChain	• Build LLM apps and simple to complex agents. • Tools, memory, chains and integrations.
LangGraph	• Graph-based, stateful workflows. • Best for long-running and complex agents.
CrewAI	• Multi-agent (team) collaboration. • Role-based agents, easy to set up.
AutoGen	• Multi-agent conversation and collaboration. • Supports code execution, tools and human-in-loop.

5. Comparison : LangChain vs LangGraph vs CrewAI vs AutoGen

Feature	LangChain	LangGraph	CrewAI	AutoGen
Main Focus	Build LLM apps & agents	Stateful agent workflows	Multi-agent team workflow	Multi-agent collaboration
Workflow Style	Chain / Modular	Graph (Nodes + Edges)	Role-based	Conversational
Multi-Agent	Limited (with tools)	Yes (via graph)	Yes (core)	Yes (core)
Best For	General use & quick agents	Complex, stateful applications	Task delegation & team agents	Research, coding & collaborative tasks

6. When to Use Which ?

- **LangChain** : When you want to quickly build an LLM application or a single agent.
- **LangGraph** : When you need complex, stateful and multi-step agent workflows.
- **CrewAI** : When you want a team of agents working together.
- **AutoGen** : When you need collaborative, multi-agent conversations (e.g., research, coding).

7. Simple Code Example (LangChain)

LangChain – Simple Agent

```python
from langchain_openai import ChatOpenAI
from langchain.agents import initialize_agent, Tool

# Define a tool
def get_time():
    return "Current time: 10:30 AM"

tools = [Tool(name="GetTime", func=get_time,
              description="Get current time")]

llm = ChatOpenAI(model="gpt-4o")
agent = initialize_agent(tools, llm, agent="zero-shot-react")

agent.run("What is the current time?")
```

@code.withnarayan

8. Real-World Use Cases

@code.withnarayan

- **Research Agent** – Collects, summarizes and analyzes information from the web.
- **Business Agent** – Automates reports, data analysis and insights.
- **Coding Agent** – Writes, runs and debugs code (e.g., AutoGen / CrewAI).
- **Multi-Agent Team** – A team of agents (researcher, writer, reviewer) to create a complete report.

9. Key Takeaways

- AI Agent frameworks make it easy to build and manage agents.
- LangChain → general & flexible, LangGraph → stateful workflows, CrewAI → multi-agent teams, AutoGen → collaborative agents.
- Choose the framework based on task complexity and workflow needs.

Practice Questions (MCQs)

1. Which framework is best for graph-based, stateful agent workflows ?
   (a) LangChain (b) LangGraph
   (c) CrewAI (d) AutoGen

2. Which framework is primarily designed for multi-agent (AI team) collaboration ?
   (a) LangChain (b) LangGraph
   (c) CrewAI (d) AutoGen

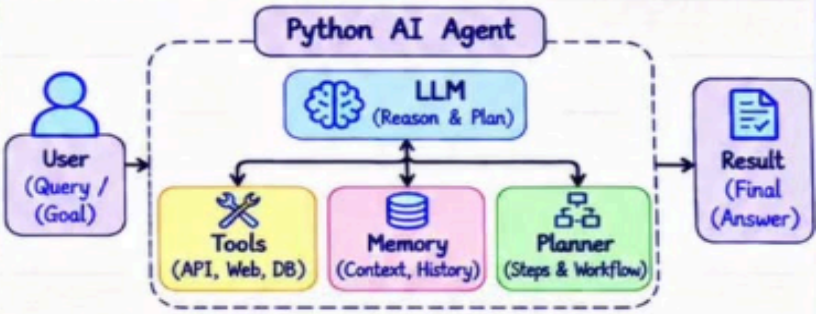
11. BUILDING AI AGENTS USING PYTHON

1. What is an AI Agent in Python?

- An AI Agent is a Python program that uses an **LLM**, **tools** and **memory** to **perceive** a goal, **plan actions** and execute tasks automatically.
- We can build AI Agents in Python using frameworks like LangChain, LangGraph, CrewAI or AutoGen.

@code.withnarayan

2. High-Level Architecture



3. Steps to Build an AI Agent (Python)

1. **Setup Environment** → Install Python and required libraries (e.g., LangChain, OpenAI).
2. **Configure LLM** → Use an LLM (e.g., GPT) with API key.
3. **Add Tools** → Integrate external tools (e.g., web search, calculator, database).
4. **Add Memory** → Keep conversation history (e.g., ConversationBufferMemory).
5. **Create Agent** → Define the agent with tools and a reasoning style (e.g., ReAct).
6. **Run & Test** → Give a goal/query and verify the agent's response.

4. Python Code Example (LangChain)

Build a Simple AI Agent

```python
from langchain_openai import ChatOpenAI
from langchain.agents import AgentExecutor, create_react_agent
from langchain.tools import Tool

# 1. Define a tool
def get_time():
    from datetime import datetime
    return datetime.now().strftime("%I:%M %p")

# 2. Create tool list
tools = [Tool(name="GetTime", func=get_time,
              description="Get current time.")]

# 3. Initialize LLM
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)

# 4. Create agent (ReAct style)
agent = create_react_agent(llm, tools)
agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True)

# 5. Run the agent
response = agent_executor.invoke({"input": "What is the current time?"})
print(response["output"])
```

@code.withnarayan

5. Code Explanation

Code Part	Purpose
Tool (get_time)	A custom tool that returns current time.
tools = [...]	List of tools that the agent can use.
ChatOpenAI(...)	Connects to an LLM (e.g., GPT-4o-mini).
create_react_agent()	Creates a ReAct-style agent (Reason + Act).
AgentExecutor(...)	Runs the agent and manages the workflow.
invoke({"input": ...})	Sends the user query and gets the result.

6. Output (Example)

User Input :
What is the current time?

Agent Output :
The current time is 10:30 AM.

7. Real-World Use Cases

- **Research Agent** – Search, read and summarize information.
- **Productivity Agent** – Manage tasks, emails and schedules.
- **Coding Agent** – Write, run and debug code.
- **Data Agent** – Fetch, analyze and create reports.

@code.withnarayan

8. Key Takeaways

- Python is a popular and flexible language to build AI Agents.
- Use an LLM for reasoning, tools for real-world actions and memory for context.
- Frameworks like LangChain, LangGraph, CrewAI and AutoGen make agent development easier.
- Start with a simple agent and then add more tools, memory and workflows.

@code.withnarayan

9. Practice Questions (MCQs)

1. Which Python framework is widely used to build AI Agents ?
   (a) NumPy (b) LangChain
   (c) Pandas (d) Matplotlib

2. In the given code, what is the role of AgentExecutor ?
   (a) To create an LLM model
   (b) To run the agent and manage its workflow
   (c) To store conversation history
   (d) To define a custom tool

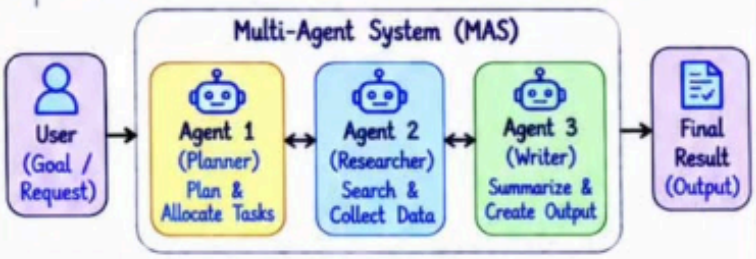
12. MULTI-AGENT SYSTEMS & COLLABORATION

1. What is a Multi-Agent System (MAS) ?

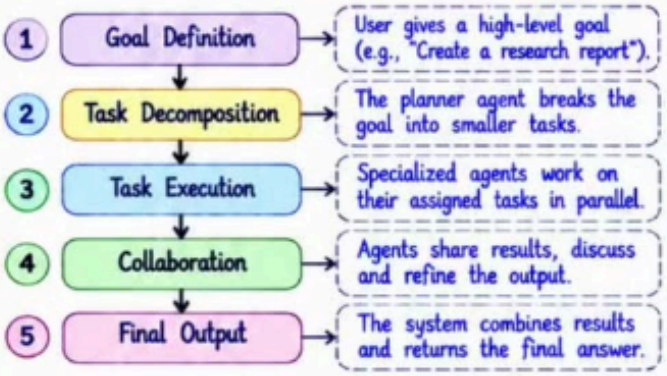
- A Multi-Agent System (MAS) is a system of multiple AI agents that work together to achieve a common goal.
- Each agent has a specific role, capability and can communicate, coordinate and collaborate with other agents.

@code.withnarayan

2. Architecture of a Multi-Agent System



3. How Multi-Agent Collaboration Works ?



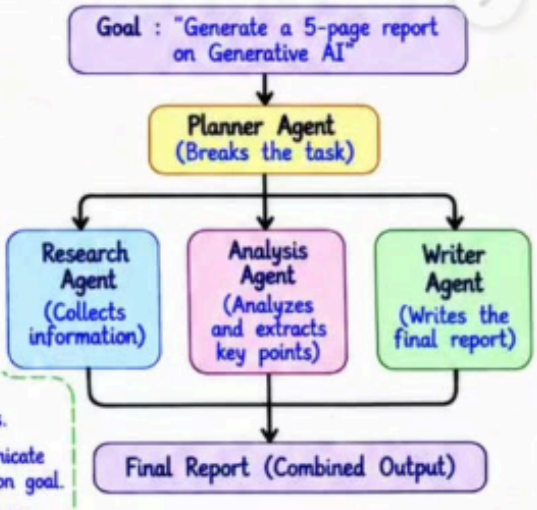
4. Key Collaboration Patterns

Pattern	Description
Hierarchical	• One main (manager) agent assigns tasks to sub-agents.
Sequential	• Agents work in a fixed order (one after another).
Parallel	• Multiple agents work at the same time on different tasks.
Peer-to-Peer	• Agents collaborate as equals and share information directly.
Debate / Reflect	• Agents discuss, critique and improve each other's results.

5. MAS vs Single Agent (Comparison)

Aspect	Single Agent	Multi-Agent System (MAS)
Agents	One agent	Multiple agents
Task Handling	Handles all tasks	Tasks are divided among agents
Complexity	Simple tasks	Complex, multi-step tasks
Efficiency	Limited (depends on one agent)	Higher (parallel & specialized work)
Use Case	Q&A, simple automation	Research, report generation, project management
Example	Single chatbot	Team of agents (Planner + Researcher + Writer)

6. Example : Research Report using MAS



@code.withnarayan

7. Real-World Use Cases

- **Research Assistant** – Multiple agents for search, analysis and report writing.
- **Software Development** – Agents for planning, coding, testing and documentation.
- **Business Analysis** – Agents to collect data, analyze and generate insights.
- **Customer Support** – Agents for query classification, response and escalation.

8. Key Takeaways

- Multi-Agent Systems use multiple specialized agents.
- Agents collaborate, communicate and work towards a common goal.
- Different collaboration patterns (e.g., hierarchical, parallel, sequential) can be used.
- MAS is useful for complex, real-world and multi-step tasks.

9. Practice Questions (MCQs)

1 What is a Multi-Agent System (MAS) ?

(a) A system with a single AI agent
(b) A system of multiple agents working together
(c) A chatbot with memory
(d) A model for only data analysis

2 Which collaboration pattern allows multiple agents to work at the same time ?

(a) Sequential (b) Hierarchical
(c) Parallel (d) Debate

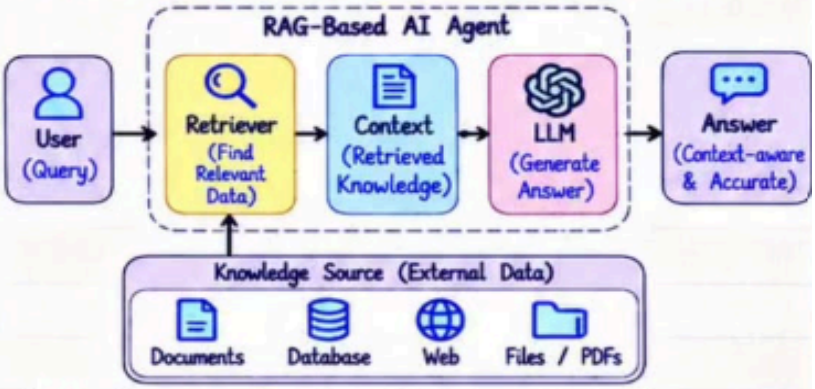
13. RAG-BASED AI AGENTS

1. What is a RAG-Based AI Agent?

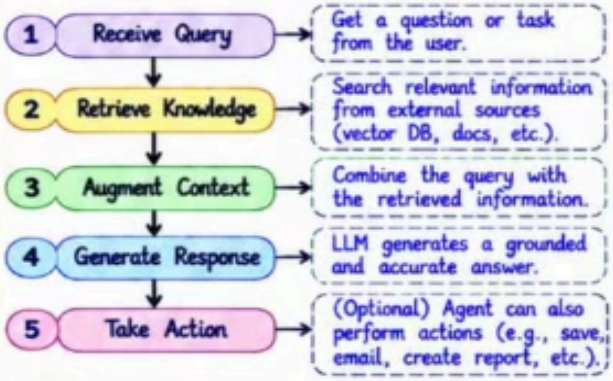
- A RAG (Retrieval-Augmented Generation) based AI Agent combines a LLM with an external knowledge source (e.g., documents, web, database) to give accurate, up-to-date and context-aware answers.
- It first retrieves relevant information and then uses the LLM to generate a final, natural language response.

@code.withnarayan

2. RAG Architecture (AI Agent with RAG)



3. Working of a RAG-Based Agent



4. Components of a RAG-Based Agent

Component	Role / Description
LLM	• Generates the final natural language answer using the retrieved context.
Retriever	• Finds the most relevant information from the knowledge source.
Embedding Model	• Converts text into vectors for semantic search (e.g., OpenAI, HuggingFace).
Vector Database	• Stores document embeddings for fast and accurate retrieval (e.g., Chroma, FAISS, Pinecone).
Knowledge Source	• External data such as PDFs, docs, web pages, Notion, database, etc.

@code.withnarayan

5. RAG vs Normal LLM (Comparison)

Aspect	Normal LLM	RAG-Based AI Agent
Knowledge	Limited to training data	Uses external, updated data
Answer Accuracy	May be outdated or wrong	More accurate and reliable
Domain Specific	Not always good	Very good (e.g., company docs)
Hallucination	Higher (can make up facts)	Lower (uses real retrieved data)
Use Case	General questions (chat)	Domain-specific, real-time, enterprise and knowledge-based tasks

6. Simple Example (Code – LangChain)

RAG Agent – Python (LangChain)

```python
from langchain_community.document_loaders import PyPDFLoader
from langchain_community.vectorstores import Chroma
from langchain_openai import ChatOpenAI
from langchain_chains import RetrievalQA

# Load and index a PDF
docs = PyPDFLoader("notes.pdf").load()
vector_db = Chroma.from_documents(docs)
# Create RAG Agent (QA Chain)
llm = ChatOpenAI(model="gpt-4o-mini")
rag_agent = RetrievalQA.from_chain_type(
    llm=llm, retriever=vector_db.as_retriever()
)
# Ask a question
print(rag_agent.invoke({"query": "What is RAG?"})
```

@code.withnarayan

7. Real-World Use Cases

- **Document Q&A** – Ask questions on PDFs, research papers or company docs.
- **Customer Support Agent** – Answer user queries using product documentation.
- **Learning Assistant** – Explain concepts using course material or notes.
- **Enterprise Search** – Get answers from internal company knowledge base (Wiki, Notion, etc.).

8. Key Takeaways

- RAG combines retrieval of relevant data with LLM generation.
- It reduces hallucination and improves accuracy.
- Uses components like retriever, embeddings, vector DB and LLM.
- Useful for domain-specific, real-time and knowledge-grounded AI Agents.

9. Practice Questions (MCQs)

1. What does RAG stand for ?
   (a) Retrieval-Augmented Generation
   (b) Real-time AI Generation
   (c) Retrieval and Generation Model
   (d) Reinforced Agent Generation

2. Which component is used to store document embeddings in a RAG system ?
   (a) LLM (b) Vector Database
   (c) Planner (d) Prompt Template

14. AGENT EVALUATION, SECURITY & RESPONSIBLE AI

1. Why Evaluate, Secure & Use Responsible AI ?

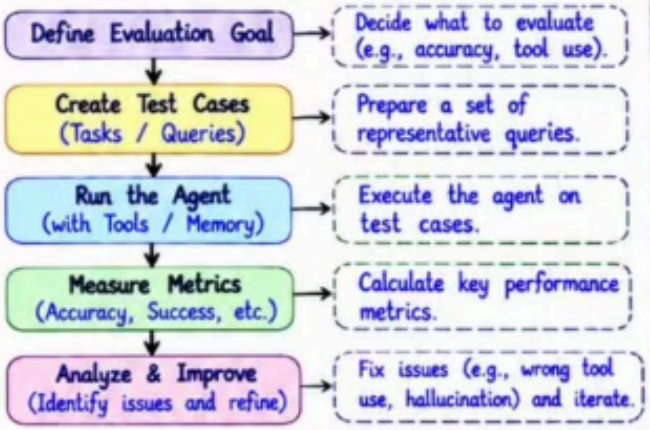
- AI Agents are powerful and can take actions, use tools and access data.
- Therefore, we must ① evaluate their performance, ② ensure security and ③ follow responsible AI practices for safe, fair and trustworthy use.

@code.withnarayan

2. Agent Evaluation – Key Metrics

Metric	Meaning (What it Measures)
Accuracy	Correctness of the agent's final answer.
Task Success Rate	% of tasks completed successfully.
Faithfulness	Whether the answer is grounded in the given data.
Tool Use Accuracy	Correct selection and execution of tools.
Latency (Time)	Time taken to complete a task.
Cost (Tokens)	Number of tokens and overall execution cost.

3. Evaluation Process (Workflow)



4. Security in AI Agents

- **Data Protection** : Keep sensitive data (PII, passwords, API keys) secure.
- **Access Control** : Give only required permissions to tools and data.
- **Secure Tool Use** : Validate inputs and prevent unauthorized actions.
- **Secrets Management** : Store API keys in environment variables / secret manager.
- **Risk Prevention** : Avoid prompt injection, data leakage and unsafe operations.

@code.withnarayan

5. Responsible AI for Agents

- **Fairness** : Avoid biased or discriminatory outputs.
- **Transparency** : Be clear about what the agent can do and its limitations.
- **Privacy** : Respect user data and follow data protection rules.
- **Safety** : Prevent harmful, illegal or misleading outputs.
- **Accountability** : Track actions (logs) and take responsibility for outcomes.

6. Common Risks & Mitigations

Risk / Issue	Mitigation (How to Handle)
Hallucination (Wrong Answer)	Use RAG, source verification and evaluation.
Prompt Injection (Manipulation)	Validate inputs, use filters and guardrails.
Data Leakage	Avoid sharing sensitive data, use access control.
Tool Misuse	Restrict tool permissions and validate actions.
Bias in Output	Use diverse data, follow responsible AI guidelines.
High Cost	Optimize prompts, set token limits and use caching.

7. Best Practices (Checklist)

- ✅ Define clear evaluation metrics and test regularly.
- ✅ Use guardrails (input/output validation).
- ✅ Follow security best practices (access control, secrets).
- ✅ Design for responsible AI (fair, safe, transparent).
- ✅ Monitor in production (logs, errors, cost, performance).

8. Key Takeaways

- ✅ Evaluate agents using clear and measurable metrics.
- ✅ Security is essential to prevent misuse and data risks.
- ✅ Responsible AI ensures fair, safe and trustworthy agents.
- ✅ Use guardrails, monitoring and continuous improvement.

9. Practice Questions (MCQs)

1 Which metric measures the success of an AI Agent in completing a task ?

(a) Latency (b) Task Success Rate
(c) Bias Score (d) Token Count

2 Which of the following is a responsible AI practice ?

(a) Using unverified data (b) Ignoring user privacy
(c) Ensuring fairness and transparency
(d) Allowing unrestricted tool access

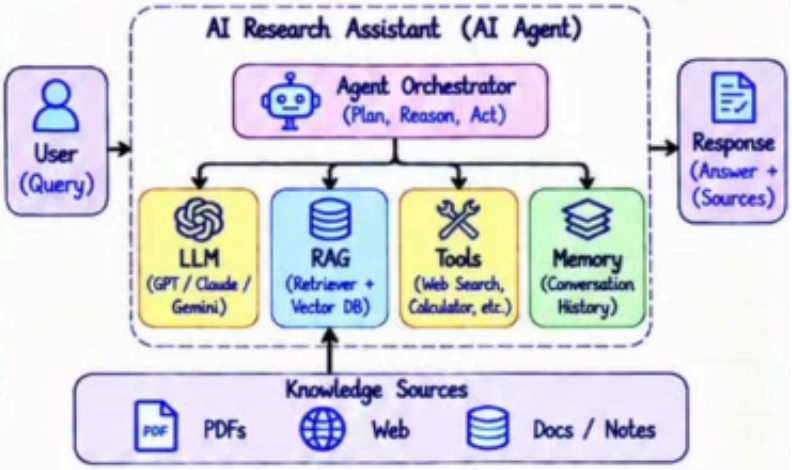
15. REAL-WORLD AI AGENT PROJECT & INTERVIEW REVISION

1. Project Overview – AI Research Assistant

- **Goal** : Build an AI Agent that can search, read, summarize and answer questions from real-world documents (PDFs / Web / Articles).
- **Features :**
  - ✓ Natural language Q&A
  - ✓ RAG (retrieve from your documents)
  - ✓ Use tools (web search, calculator)
  - ✓ Keep conversation memory
  - ✓ Source-cited answers

@code.withnarayan

2. System Architecture



3. Tech Stack

Layer	Technology
Language	Python
Framework	LangChain (or LangGraph)
LLM	OpenAI / Gemini / Claude
Vector DB	Chroma / FAISS / Pinecone
Tools	Tavily (Web Search), Calculator
App (Optional)	Streamlit / FastAPI
Data	PDFs, Docs, Web Articles

4. Project Workflow (Step-by-Step)

1. **Setup** → Install Python and required libraries (langchain, openai, chromadb, streamlit, etc.).
2. **Load Data** → Ingest documents (PDFs) and create vector store (embeddings).
3. **Define Tools** → Add tools like web search and calculator.
4. **Build Agent** → Create an agent with LLM + RAG + Tools + Memory.
5. **Build UI** → (Optional) Create a simple UI using Streamlit.
6. **Run & Test** → Ask real questions and get grounded answers with sources.

5. Minimal Code (LangChain)

AI Research Assistant (Core Code)

```python
from langchain_openai import ChatOpenAI
from langchain_community.vectorstores import Chroma
from langchain_community.document_loaders import PyPDFLoader
from langchain.chains import RetrievalQA

# Load documents
loader = PyPDFLoader("notes.pdf")
docs = loader.load()

# Create vector store
vector_db = Chroma.from_documents(docs)
retriever = vector_db.as_retriever()

# Create RAG agent
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
qa = RetrievalQA.from_chain_type(llm=llm,
                retriever=retriever)

# Ask a question
qa.invoke({"query": "What is an AI Agent?"})
```

@code.withnarayan

6. Sample Output

Q : What is an AI Agent?

A : An AI Agent is an autonomous system that uses an LLM, tools and memory to perceive, plan and take actions to achieve a goal.

Sources : notes.pdf (p. 3), article.com

7. Interview Revision (Important Questions)

1. What is an AI Agent ?
2. How is an AI Agent different from a chatbot ?
3. Explain the role of tools and function calling.
4. What is RAG and why is it used ?
5. What is the difference between LangChain and LangGraph ?
6. How do you add memory to an AI Agent ?
7. How do you evaluate an AI Agent ?

8. Key Takeaways

- ✓ AI Agents combine LLM + Tools + Memory + Planning.
- ✓ RAG makes agents more accurate and domain-specific.
- ✓ Frameworks like LangChain / LangGraph help build production-ready agents.
- ✓ Focus on evaluation, security and responsible AI.

9. Quick Revision – FAQs (One-Liners)

Topic	One-Liner Answer
AI Agent	An autonomous system that perceives, plans and takes actions.
Tools	External capabilities (e.g., web search, calculator, API).
Function Calling	Letting the LLM call predefined functions with structured input.
Memory	Stores conversation context (short-term or long-term).
RAG	Combines retrieval of external knowledge with LLM generation.
LangChain	Framework to build LLM apps and agents quickly.
LangGraph	Builds stateful, complex and multi-agent workflows.